Django Database & ORM — Full Notes A to Z

Django 6.0 | English Explanation and Python Examples

Part 1: Introduction to Database and Django ORM

1. What Is a Database?

Student names and email addresses Course names Student course enrollments Examination results

2. What Is ORM?

3. Django Database Configuration

A database is a system for storing, retrieving, updating, and deleting structured data.

For example, a Student Management System may store:

ORM stands for Object-Relational Mapping. Django ORM allows developers to work with relational databases using Python code instead of writing SQL for every operation.

SQL:

```python
SELECT * FROM students WHERE age >= 18;
```

Django ORM:

```
Student.objects.filter(age__gte=18)
```

The DATABASES setting in settings.py configures database connections.

SQLite example:

```python
DATABASES = {
"default": {
"ENGINE": "django.db.backends.sqlite3",
"NAME": BASE_DIR / "db.sqlite3",
}
}
```

Common database backends include SQLite, PostgreSQL, MySQL, and Oracle.

4. What Are Migrations?

makemigrations : Creates migration files based on model changes. migrate : Applies migrations to the database.

Part 2: Models

5. What Is a Model?

The availability of particular features depends on the Django version, database driver, and backend. Migrations are Django's mechanism for applying model changes to the database schema.

```python
python manage.py makemigrations
python manage.py migrate
Additional commands:
python manage.py showmigrations
python manage.py sqlmigrate app_name 0001
```

showmigrations displays migration status. sqlmigrate displays the SQL generated for a particular migration.

Remember: Changing a model does not automatically change the database schema. You generally need to create and apply migrations. Official documentation: https://docs.djangoproject.com/en/6.0/topics/db/models/ A Django Model is a Python class that generally corresponds to a database table.

```python
from django.db import models
class Student(models.Model):
name = models.CharField(max_length=100)
email = models.EmailField(unique=True)
age = models.PositiveIntegerField()
def __str__(self):
return self.name
Here:
```

Student : Model class name , email , age : Model fields A Student instance: One student record __str__() : Returns a readable representation of the object

6. Important Model Fields

Field Purpose CharFie ld Short or fixedmaximumlength text TextFie ld Long text Integer Field Integer values Positiv eInteger Field Positive integer values Boolean Field True/Fals e values EmailFi eld Email addresses URLFiel d URLs DateFie ld Dates

DateTim eField Dates and times TimeFie ld Times Decimal Field Precise decimal values FloatFi eld Floatingpoint values UUIDFie ld UUID values JSONFie ld JSONcompatibl e data FileFie ld File path/refer ence ImageFi eld Image path/refer ence Foreign Key Many-toone relationshi p OneToOn eField One-toone

7. Common Field Options

null=True blank=True default unique=True relationshi p ManyToM anyFiel d Many-tomany relationshi p

Example:

```python
class Product(models.Model):
name = models.CharField(max_length=150)
description = models.TextField(blank=True)
price = models.DecimalField(max_digits=10, decimal_places=2)
stock = models.PositiveIntegerField(default=0)
is_available = models.BooleanField(default=True)
created_at = models.DateTimeField(auto_now_add=True)
updated_at = models.DateTimeField(auto_now=True)
```

For financial values, DecimalField is generally preferable to FloatField . Allows the database to store NULL . age = models.IntegerField(null=True, blank=True) Allows a field to be empty during Django validation. Defines a default value. is_active = models.BooleanField(default=True) Prevents duplicate values in the field. email = models.EmailField(unique=True) db_index=True choices primary_key=True

8. Difference Between null and blank

Requests a database index for a field. email = models.EmailField(db_index=True) Restricts a field to a predefined set of choices.

```python
class Student(models.Model):
class Status(models.TextChoices):
ACTIVE = "active", "Active"
INACTIVE = "inactive", "Inactive"
name = models.CharField(max_length=100)
status = models.CharField(
```

max_length=10,
choices=Status,
default=Status.ACTIVE,
)
Get the display label:
```
student.get_status_display() Defines the primary key used to identify a record.
```python
import uuid
student_id = models.UUIDField(
primary_key=True,
default=uuid.uuid4,
editable=False,
)
Django can create a primary key automatically. If you define a custom primary key,
```
Django generally does not add another id field.
9. Model Meta Options
db_table : Database table name ordering : Default ordering verbose_name : Singular display name Option Purpose null=Tr ue Allows NULL values in the database blank=T rue Allows empty values during validation
For example:
description = models.TextField(blank=True) For text fields, an empty string is generally preferred over NULL unless nullable text has a specific purpose. The inner Meta class defines model metadata.
```python
class Product(models.Model):
name = models.CharField(max_length=100)
price = models.DecimalField(max_digits=10, decimal_places=2)
class Meta:
db_table = "store_product"
ordering = ["name"]
verbose_name = "Product"

verbose_name_plural = "Products"
Important options:
```
verbose_name_plural : Plural display name constraints : Database-level constraints
indexes : Index configuration unique_together : Older composite uniqueness option;
UniqueConstraint is generally preferred in new code
10. Constraints
Part 3: Model Relationships
Constraints help enforce data integrity at the database level.
```python
class Product(models.Model):
name = models.CharField(max_length=100)
price = models.DecimalField(max_digits=10, decimal_places=2)
class Meta:
constraints = [
models.CheckConstraint(
condition=models.Q(price__gte=0),
name="product_price_nonnegative",
)
]
```
This creates a rule preventing negative prices when the database backend supports the constraint.
To enforce uniqueness across multiple fields:
```python
class Enrollment(models.Model):
student = models.ForeignKey("Student", on_delete=models.CASCADE)
course = models.ForeignKey("Course", on_delete=models.CASCADE)
class Meta:
constraints = [
models.UniqueConstraint(
fields=["student", "course"],
name="unique_student_course",

)
]
```
Official documentation: https://docs.djangoproject.com/en/6.0/topics/db/models/
11. One-to-Many / Many-to-One
CASCADE : Deletes related objects when the referenced object is deleted. PROTECT : Prevents deletion when protected related objects exist. RESTRICT : Restricts deletion when related objects exist, subject to Django's deletion rules. SET_NULL : Sets the foreign key to NULL ; requires null=True . SET_DEFAULT : Sets the foreign key to its default value. SET(...) : Sets a specified value or callable result. DO_NOTHING : Django takes no deletion action; a database constraint may
raise an error.
12. One-to-One Relationship
One department can have many students, while each student belongs to one department in this example.
```python
class Department(models.Model):
name = models.CharField(max_length=100)
class Student(models.Model):
name = models.CharField(max_length=100)
department = models.ForeignKey(
Department,
on_delete=models.CASCADE,
related_name="students",
)
Usage:
```
department.students.all() student.department student.department_id A ForeignKey normally requires an on_delete option.
Important on_delete options:
A user can have one profile.
```python
from django.conf import settings

from django.db import models
```
13. Many-to-Many Relationship
```python
class Profile(models.Model):
user = models.OneToOneField(
settings.AUTH_USER_MODEL,
on_delete=models.CASCADE,
related_name="profile",
)
bio = models.TextField(blank=True)
Usage:
```
profile.user user.profile Using settings.AUTH_USER_MODEL is recommended when referring to the project's user model, because the project may use a custom user model.
A student can enroll in multiple courses, and each course can have multiple students.
```python
class Course(models.Model):
title = models.CharField(max_length=100)
class Student(models.Model):
name = models.CharField(max_length=100)
courses = models.ManyToManyField(
Course,
related_name="students",
)
Usage:
```
student.courses.add(course1, course2) student.courses.remove(course1) student.courses.clear() student.courses.set([course1, course2]) student.courses.all() course.students.all()
14. Through Model
15. Reverse Relationships
```

Django normally creates an intermediate table for a many-to-many relationship. Use a through model when a relationship needs additional data.

```python
class Student(models.Model):
name = models.CharField(max_length=100)
class Course(models.Model):
title = models.CharField(max_length=100)
students = models.ManyToManyField(
Student,
through="Enrollment",
related_name="courses",
)
class Enrollment(models.Model):
student = models.ForeignKey(Student, on_delete=models.CASCADE)
course = models.ForeignKey(Course, on_delete=models.CASCADE)
enrolled_at = models.DateTimeField(auto_now_add=True)
grade = models.CharField(max_length=5, blank=True)
```

The Enrollment model stores additional information such as the enrollment date and grade. Without related_name , Django generally uses_set for reverse relationships.

```python
class Book(models.Model):
author = models.ForeignKey("Author", on_delete=models.CASCADE)
Usage:
```

```
author.book_set.all()
If related_name="books" is defined:
author.books.all()
```

16. Recursive Relationships

Part 4: Making Queries and QuerySets

17. What Is a QuerySet?

18. Retrieve All Records

19. Filter Records

20. Exclude Records

A model can have a relationship with itself.

```python
class Employee(models.Model):
name = models.CharField(max_length=100)
manager = models.ForeignKey(
"self",
null=True,
blank=True,
on_delete=models.SET_NULL,
related_name="team_members",
)
```

Official documentation: https://docs.djangoproject.com/en/6.0/topics/db/queries/ A QuerySet is a collection-like object representing the results of a database query.

Student.objects.all()

QuerySets are generally lazy: creating one does not immediately execute the query. Evaluation may occur when iterating, converting it to a list, calling len() , evaluating its truth value, or using certain terminal operations.

Student.objects.all()

Returns records matching specified conditions.

```python
Student.objects.filter(age=20)
Student.objects.filter(age__gte=18)
Student.objects.filter(name__startswith="A")
```

Excludes records matching specified conditions.

Student.objects.exclude(age__lt=18)

21. Get a Single Object

Student.DoesNotExist

Student.MultipleObjectsReturned

22. QuerySet Lookups

student = Student.objects.get(id=1)

Possible exceptions:

Example with exception handling:

try:

student = Student.objects.get(id=1)

except Student.DoesNotExist:

student = None

Use get() when the query is expected to return exactly one object. Lookup Meaning exact Exact match iexact Caseinsensitive exact match contain s Contains a substring icontai ns Caseinsensitive substring match startsw ith Starts with a value istarts with Caseinsensitive prefix match endswit h Ends with a value gt Greater than gte Greater than or equal to lt Less than lte Less than or equal to in Matches a collection range Falls within a range isnull Checks whether a value is NULL

23. AND Conditions

24. OR Conditions with Q Objects

& : AND | : OR ~ : NOT year Filters by year date Filters by date

Examples:

```python
Student.objects.filter(age__gt=18)
Student.objects.filter(age__range=(18, 25))
Student.objects.filter(name__icontains="faruk")
Student.objects.filter(id__in=[1, 2, 3])
Student.objects.filter(birth_date__year=2000)
Student.objects.filter(email__isnull=True)
```

Multiple conditions in one filter() are combined using AND.

```python
Student.objects.filter(age__gte=18, is_active=True)
from django.db.models import Q
Student.objects.filter(
Q(age__lt=18) | Q(age__gt=60)
)
```

Operators:

Example:

```
Student.objects.filter(
Q(name__icontains="omar") |
Q(email__icontains="omar")
)
```

25. Ordering
26. Limit and Offset
27. Values and Values List
28. Distinct Results
29. Exists, Count, First, and Last

```python
Student.objects.order_by("name")
Student.objects.order_by("-age")
```

The - prefix indicates descending order.

Ordering by multiple fields:

```python
Student.objects.order_by("department__name", "-age")
Student.objects.all()[:10]
Student.objects.all()[10:20]
```

These operations are useful for limiting results and implementing pagination. Negative indexing and ordinary reverse Python slicing are not supported by QuerySet slicing. After slicing, further filtering or ordering may also be restricted.

Retrieve only selected fields:

Student.objects.values("id", "name")

This returns dictionary-like records.

Student.objects.values_list("name", flat=True)

This returns a flat iterable of names. Use distinct() to remove duplicate results.

Student.objects.values("department_id").distinct()

The fields used for distinct selection and the effects of ordering can depend on the database backend.

```python
```

Student.objects.count()
Student.objects.exists()
```
count() : Returns the number of records. exists() : Checks whether a matching record exists. first() : Returns the first object or None . last() : Returns the last object or None .
### 30. Get or Create
student : The retrieved or newly created object created : Whether a new object was created
### 31. Update or Create
### 32. Create Records
```python
Student.objects.first()
Student.objects.last()
```
Use exists() when you only need to know whether a result exists. Consider whether a separate query is necessary if you will later use the full QuerySet anyway. student, created = Student.objects.get_or_create(
email="student@example.com",
defaults={
"name": "Rahim",
"age": 20,
},
)
Database uniqueness constraints are important for avoiding race conditions. student, created = Student.objects.update_or_create(
email="student@example.com",
defaults={
"name": "Rahim",
"age": 21,
},
)
student = Student.objects.create(
name="Rahim",
```

email="rahim@example.com",
age=20,
)
33. Update Records
34. Delete Records
35. F Expressions
Alternatively:
student = Student(
name="Rahim",
email="rahim@example.com",
age=20,
)
student.save()
Update multiple records at once:
Student.objects.filter(age=20).update(age=21)
QuerySet.update() performs a direct SQL update. It does not call each instance's save() method or trigger the usual model save signals.
Update one object:
student = Student.objects.get(pk=1)
student.age = 22 student.save()
```python
Student.objects.filter(is_active=False).delete()
Delete one object:
```
student.delete() Always verify the filter before deleting data. Calling all().delete() can delete every record in the table. F() expressions refer to database field values and allow database-side calculations.
```python
from django.db.models import F
Product.objects.filter(pk=1).update(
stock=F("stock") - 1
)
```

36. Conditional Queries with Case and When

37. Bulk Operations

Part 5: Aggregation and Annotation

38. What Is Aggregation?

This avoids reading a value into Python and then writing it back as a separate operation.

```python
from django.db.models import Case, When, Value, CharField
students = Student.objects.annotate(
level=Case(
When(age__lt=18, then=Value("Minor")),
default=Value("Adult"),
output_field=CharField(),
)
)
Student.objects.bulk_create([
Student(name="A", email="a@example.com", age=20),
Student(name="B", email="b@example.com", age=21),
```

])

```python
Student.objects.bulk_update(
students,
["age"],
)
```

Bulk operations can be efficient for large numbers of records. However, they do not follow the normal instance-level save() and signal behavior. Generated primary keys and other details can also vary by backend and operation. Official documentation: https://docs.djangoproject.com/en/6.0/topics/db/aggregation/ Aggregation calculates summary values across multiple records.

```python
from django.db.models import Avg, Count, Max, Min, Sum
Example:
```

Student.objects.aggregate(Avg("age"))
```

### 39. Common Aggregate Functions

### 40. What Is Annotation?

Function Purpose Count() Counts records Sum() Calculates a total Avg() Calculates an average Max() Finds the maximum Min() Finds the minimum

Example:

```python
Product.objects.aggregate(
total_products=Count("id"),
total_stock=Sum("stock"),
average_stock=Avg("stock"),
highest_stock=Max("stock"),
lowest_stock=Min("stock"),
)
```

aggregate() generally returns a dictionary. annotate() adds calculated values to individual objects or groups.

```python
from django.db.models import Count
departments = Department.objects.annotate(
student_count=Count("students")
```

41. Aggregation vs Annotation

42. Grouping Results

43. Order of Filter and Annotation

)

```
for department in departments:
print(department.name, department.student_count)
```

aggrega te() annotat e() Produces an overall summary Adds summary values to objects or groups Generally returns a dictionary Returns an annotated QuerySet

Example:

Total number of students
```

Example:

Number of students per departme nt

```python
Student.objects.values("department_id").annotate(
total_students=Count("id")
)
```

This calculates the number of students in each department.

44. Conditional Aggregation

45. Distinct Count

Part 6: Database Search

46. Basic Text Search

47. Search Across Multiple Fields

Publisher.objects.annotate(
book_count=Count("book")
).filter(book_count__gt=5) Changing the order of filter() and annotate() can change the results. For complex aggregation, inspect the generated SQL and joins.

```python
from django.db.models import Count, Q
Department.objects.annotate(
active_students=Count(
"students",
filter=Q(students__is_active=True),
)
)
Department.objects.annotate(
student_count=Count("students", distinct=True)
)
```

distinct=True can be useful when joins create duplicate rows. Understand the query structure before using it. Official documentation: https://docs.djangoproject.com/en/6.0/topics/db/search/

Student.objects.filter(name__icontains="rahim")

The case sensitivity of contains and icontains can depend on the database backend and collation.

```python
from django.db.models import Q
```

48. PostgreSQL Full-Text Search

Part 7: Managers

```python
Student.objects.filter(
Q(name__icontains="rahim") |
Q(email__icontains="rahim")
)
```

When using PostgreSQL, Django provides SearchVector , SearchQuery , and SearchRank .

```python
from django.contrib.postgres.search import (
SearchQuery,
SearchVector,
SearchRank,
)
articles = Article.objects.annotate(
search=SearchVector("title", "body")
).filter(
search=SearchQuery("django database")
)
```

Rank results by relevance:

```
query = SearchQuery("django database")
articles = (
Article.objects
.annotate(
rank=SearchRank(
SearchVector("title", "body"),
```

query,
)
)
.filter(rank__gt=0)
.order_by("-rank")
)
```

These PostgreSQL-specific search APIs require the PostgreSQL backend.

### 49. What Is a Manager?

### 50. Custom Manager

### 51. Custom Manager Methods

Official documentation: https://docs.djangoproject.com/en/6.0/topics/db/managers/ A Manager provides an interface for accessing a model's database queries.

Student.objects.all()

Here, objects is the conventional default manager name.

```python
class ActiveStudentManager(models.Manager):
def get_queryset(self):
return super().get_queryset().filter(is_active=True)
class Student(models.Model):
name = models.CharField(max_length=100)
is_active = models.BooleanField(default=True)
objects = models.Manager()
active_students = ActiveStudentManager()
```

Usage:

```
Student.objects.all()
Student.active_students.all()
```

Custom managers help organize reusable query logic.

```python
class StudentManager(models.Manager):
def adults(self):
return self.get_queryset().filter(age__gte=18)
```
```

class Student(models.Model):
name = models.CharField(max_length=100)
age = models.PositiveIntegerField()
objects = StudentManager()
```

### 52. QuerySet.as_manager()

53. _default_manager and _base_manager

_default_manager : The model's configured default manager. _base_manager : A manager Django uses for certain internal operations and related-object access.

## Part 8: Performing Raw SQL Queries

### 54. Why Use Raw SQL?

### 55. Manager.raw()

Usage:

Student.objects.adults()

Expose reusable QuerySet methods through a Manager.

```python
class StudentQuerySet(models.QuerySet):
def active(self):
return self.filter(is_active=True)
def adults(self):
return self.filter(age__gte=18)
class Student(models.Model):
name = models.CharField(max_length=100)
age = models.PositiveIntegerField()
is_active = models.BooleanField(default=True)
objects = StudentQuerySet.as_manager()
Student.objects.active().adults()
```

When implementing soft deletion or filtered managers, understand the distinction. A base manager that filters out records can interfere with related-object access. Official documentation: https://docs.djangoproject.com/en/6.0/topics/db/sql/ Django ORM handles most common database operations. Raw SQL can be useful for complex or
```

database-specific operations. %s is used as a parameter placeholder. Parameters are supplied separately in a list. User input is not directly concatenated into the SQL string.

56. connection.cursor()

57. Preventing SQL Injection

students = Student.objects.raw(
"SELECT * FROM school_student WHERE age >= %s",
[18],
)
for student in students:
print(student.name)

Important points:

raw() generally returns an iterable of model instances. The SQL result must include the model's primary key.

For more direct SQL execution:

```python
from django.db import connection
with connection.cursor() as cursor:
cursor.execute(
"SELECT COUNT(*) FROM school_student WHERE age >= %s",
[18],
)
result = cursor.fetchone()
```

result is generally a tuple. For multiple databases, use a connection from connections["alias"] .

Unsafe approach:

query = f"SELECT * FROM school_student WHERE name = '{name}'"

This can expose the application to SQL injection.

Safer approach:

cursor.execute(
"SELECT * FROM school_student WHERE name = %s",

Part 9: Database Transactions

58. What Is a Transaction?

59. transaction.atomic()

60. Atomic Function Decorator

61. ATOMIC_REQUESTS

[name],

)

Use parameterized queries. Table and column names cannot generally be bound as ordinary value parameters. If they must be dynamic, use a trusted allowlist. Official documentation: https://docs.djangoproject.com/en/6.0/topics/db/transactions/ A transaction is a logical unit of one or more database operations. For example, a bank transfer must debit one account and credit another account consistently.

```python
from django.db import transaction
with transaction.atomic():
account1.balance -= 100
account1.save()
account2.balance += 100
account2.save()
```

When the transaction block completes successfully, changes are committed. Database errors can cause the transaction to roll back. This example demonstrates the concept. A real financial transfer also requires balance validation, concurrency protection, and database-level integrity.

```python
from django.db import transaction
@transaction.atomic
def create_order():
# Related database operations
...
```

62. transaction.on_commit()

63. Nested Atomic Blocks

Example database configuration:

```python
```

```
DATABASES = {
"default": {
# Backend settings...
"ATOMIC_REQUESTS": True,
}
}
```

This can wrap the execution of each view request in a transaction for the relevant database. It does not automatically include every stage of request processing. Middleware and template response processing, for example, can be outside the transaction boundary.

Run a callback after a successful transaction commit:

```python
from django.db import transaction
with transaction.atomic():
order = Order.objects.create(...)
transaction.on_commit(
lambda: send_order_notification(order.pk)
)
```

The callback does not run if the transaction rolls back. This is useful for sending emails, updating caches, or triggering external side effects after a successful commit.

```python
with transaction.atomic():
order = Order.objects.create(...)
with transaction.atomic():
order_item = OrderItem.objects.create(...)
```

Nested atomic blocks generally use savepoints when supported. If the outer transaction rolls back, changes from a successful inner block can also be rolled back.

64. Exception Handling

65. Important Limitations

A transaction generally operates on one database connection. Changes across multiple database aliases are not automatically committed as one atomic transaction. External API calls and emails cannot be rolled back by a database transaction. For concurrency-sensitive operations, consider select_for_update() , constraints, or atomic updates.

Part 10: Multiple Databases

66. What Are Multiple Databases?

default : Main application database analytics : Analytics data legacy : An older database

67. Database Configuration

Continuing to run queries inside a transaction after a database error can cause problems.

A safer pattern:

```python
from django.db import IntegrityError, transaction
try:
with transaction.atomic():
create_unique_record()
except IntegrityError:
handle_duplicate_record()
```

The exception is caught outside the atomic() block, allowing Django to handle the failed transaction state correctly. Official documentation: https://docs.djangoproject.com/en/6.0/topics/db/multi-db/ A Django project can use multiple database connections.

For example:

```python
DATABASES = {
"default": {
```

68. using()

69. Select a Database When Saving

70. db_manager()

71. Database Router

"ENGINE": "django.db.backends.sqlite3",

"NAME": BASE_DIR / "db.sqlite3",
},
"analytics": {
"ENGINE": "django.db.backends.sqlite3",
"NAME": BASE_DIR / "analytics.sqlite3",
},
}
In production, each database requires appropriate engine, credentials, and connection settings.
```python
Student.objects.using("analytics").all()
student = Student(name="Rahim", age=20)
```
student.save(using="analytics")
Choose a database for manager methods:
```python
Student.objects.db_manager("analytics").create(
name="Rahim",
age=20,
)
```
A router determines which database should handle reads, writes, relations, and migrations.
```python
class AnalyticsRouter:
route_app_labels = {"analytics_app"}
def db_for_read(self, model, **hints):
if model._meta.app_label in self.route_app_labels:
return "analytics"
return None
```
72. Cross-Database Relationships
Part 11: Tablespaces

73. What Is a Tablespace?

```python
def db_for_write(self, model, **hints):
if model._meta.app_label in self.route_app_labels:
return "analytics"
return None
def allow_relation(self, obj1, obj2, **hints):
if (
obj1._state.db == "analytics"
or obj2._state.db == "analytics"
):
return obj1._state.db == obj2._state.db
return None
def allow_migrate(self, db, app_label, **hints):
if app_label in self.route_app_labels:
return db == "analytics"
return None
In settings.py :
DATABASE_ROUTERS = ["project.routers.AnalyticsRouter"]
This is an illustrative example. A real router needs explicit policies for all relevant apps,
```

relations, and migrations. Django does not automatically support ordinary relational foreign keys or many-to-many relationships across separate databases. Plan relationships, migrations, and transaction behavior carefully before adopting multiple databases. Official documentation: https://docs.djangoproject.com/en/6.0/topics/db/tablespaces/ A tablespace is a database storage organization concept. Some database backends use tablespaces to control where tables and indexes are stored.

Example model tablespace:

Part 12: Database Access Optimization

74. Why Is Optimization Important?

75. QuerySet Laziness

```python
```

class Student(models.Model):
name = models.CharField(max_length=100)
class Meta:
db_tablespace = "student_tablespace"
Example index tablespace:
class Student(models.Model):
email = models.EmailField(db_index=True)
class Meta:
db_tablespace = "student_tablespace"
indexes = [
models.Index(
fields=["email"],
db_tablespace="student_index_tablespace",
)
]
```

Tablespace support is backend-specific. For example, SQLite may not support this configuration in the expected way. Official documentation: https://docs.djangoproject.com/en/6.0/topics/db/optimization/ Slow database queries increase API response times and server resource usage. Before optimizing, measure query counts, execution plans, table sizes, and real workloads.
students = Student.objects.filter(age__gte=18)
This line generally does not immediately execute the query.
for student in students:
print(student.name)
The QuerySet may be evaluated during iteration.
### 76. QuerySet Caching
### 77. N+1 Query Problem
Solution: select_related()
78. prefetch_related()
Difference An evaluated QuerySet can reuse its result cache when the same QuerySet instance is used again.
students = Student.objects.all()
```

list(students) list(students) The second evaluation can reuse the cached result. Creating a separate QuerySet may execute another query.

Consider:

students = Student.objects.all()

for student in students:

print(student.department.name)

If each department is fetched separately, the code may generate many queries.

students = Student.objects.select_related("department")

select_related() generally uses SQL joins for ForeignKey and OneToOneField relationships.

For collections such as many-to-many and reverse relationships:

departments = Department.objects.prefetch_related("students")

for department in departments:

for student in department.students.all():

print(student.name)

prefetch_related() generally uses additional queries and joins the results in Python.

79. only() and defer()

select_ related(

)

prefetc h_relate d() Generally uses SQL JOIN Generally uses separate queries and Pythonside matching Best suited to ForeignKe y and OneToOn e Supports ManyToM any, reverse relationshi ps, and more Can fetch related columns in one query May execute multiple queries

Retrieve only selected fields:

```python

Student.objects.only("id", "name")

Defer a large field:

Article.objects.defer("large_content")

```

80. values() and values_list()

81. iterator()

82. explain()

83. Indexing

Too many indexes can slow write operations. Indexes consume storage. Index selection should reflect real query patterns. Accessing a deferred field later can trigger an additional query. Use these methods only when justified by actual performance measurements.

Retrieve selected data instead of full model instances:

```python
Student.objects.values("id", "name")
Student.objects.values_list("name", flat=True)
```

These methods can be useful for reporting and data serialization. Process large QuerySets without retaining the normal QuerySet result cache:

for student in Student.objects.all().iterator(chunk_size=1000):
process_student(student)

This can help with large data processing. Behavior may vary with the backend and prefetch configuration.

Inspect how the database plans to execute a query:

print(

```python
Student.objects.filter(age__gte=18).explain()
)
```

The execution plan can reveal table scans, index usage, and joins.

```python
class Student(models.Model):
email = models.EmailField(unique=True)
age = models.PositiveIntegerField(db_index=True)
```

Indexes can improve searches and filtering, but:

Composite index:

```

### 84. Atomic Updates with F()

### 85. Bulk Create and Bulk Update

86. count() , exists() , and contains()
```

Use count() when you need the count. Use exists() when you only need to know whether any matching record exists. Use contains() when checking whether a specific object belongs to a QuerySet.

```python
class Meta:
indexes = [
models.Index(fields=["department", "age"]),
]
```

The order of fields in a composite index matters.

```python
from django.db.models import F
Product.objects.filter(pk=1).update(
stock=F("stock") - 1
)
```

This reduces the risk of application-side read-modify-write races. Database constraints or conditional updates may still be needed to prevent negative stock.

```python
Student.objects.bulk_create(student_objects)
Student.objects.bulk_update(
student_objects,
["age"],
)
```

Bulk operations can be more efficient than calling save() for every record.

```python
Student.objects.count()
Student.objects.filter(age__gte=18).exists()
Student.objects.contains(student)
```

Consider whether an additional query is necessary when the results will be reused.

87. Pagination

Part 13: Database Instrumentation

88. What Is Database Instrumentation?

89. Query Logging

90. connection.execute_wrapper()

```python
from django.core.paginator import Paginator
students = Student.objects.order_by("id")
paginator = Paginator(students, 20)
page = paginator.get_page(1)
```

Use stable ordering before paginating. Official documentation: https://docs.djangoproject.com/en/6.0/topics/db/instrumentation/ Database instrumentation means monitoring database queries and connection activity. It helps analyze query execution, performance issues, and database behavior. During development, Django's SQL logging can show executed database queries.

```python
LOGGING = {
"version": 1,
"handlers": {
"console": {
"class": "logging.StreamHandler",
},
},
"loggers": {
"django.db.backends": {
"handlers": ["console"],
"level": "DEBUG",
},
},
}
```

This is a development-oriented example. In production, consider log volume, sensitive information, and performance impact.

Part 14: Fixtures

91. What Is a Fixture?

92. Fixture Formats

Monitor query execution using a wrapper:

```python
from django.db import connection
def query_wrapper(execute, sql, params, many, context):
print("SQL:", sql)
return execute(sql, params, many, context)
with connection.execute_wrapper(query_wrapper):
list(Student.objects.all())
```

Real instrumentation can record duration, errors, and query metadata. Avoid logging passwords, tokens, or personal data. Official documentation: https://docs.djangoproject.com/en/6.0/topics/db/fixtures/ A fixture is prepared data that can be loaded into a database. Fixtures are useful for development, testing, and initial sample data. Django fixtures commonly use JSON, XML, or YAML formats, subject to the available serializers and dependencies.

JSON example:

```
[
{
"model": "school.student",
"pk": 1,
"fields": {
"name": "Rahim",
"email": "rahim@example.com",
"age": 20
}
```

93. Load Fixtures

94. Dump Fixtures

95. Fixture Precautions

Check privacy before exporting production data. Primary key conflicts can occur. Foreign key and many-to-many dependencies must be handled correctly. Consider transaction

and signal behavior when loading fixtures. Never include sensitive credentials in fixtures.

Part 15: Relationship API Examples

96. Many-to-One Example

}

]

Here, school is the app label and student is the model name.

```python
python manage.py loaddata students.json
python manage.py dumpdata school.Student --indent 2
Save output to a file:
python manage.py dumpdata school.Student --indent 2 > students.json
```

Official documentation: https://docs.djangoproject.com/en/6.0/topics/db/examples/

```python
class Reporter(models.Model):
name = models.CharField(max_length=100)
class Article(models.Model):
headline = models.CharField(max_length=200)
reporter = models.ForeignKey(
Reporter,
on_delete=models.CASCADE,
related_name="articles",
)
```

97. One-to-One Example

Create records:

reporter = Reporter.objects.create(name="Rahim")
Article.objects.create(
headline="Django ORM",
reporter=reporter,
)

Forward access:

article = Article.objects.first() article.reporter

Reverse access:
reporter.articles.all()
```python
class Place(models.Model):
name = models.CharField(max_length=100)
class Restaurant(models.Model):
place = models.OneToOneField(
Place,
on_delete=models.CASCADE,
related_name="restaurant",
)
serves_pizza = models.BooleanField(default=False)
```
place = Place.objects.create(name="Central Place") restaurant =
Restaurant.objects.create(
place=place,
serves_pizza=True,
)
place.restaurant restaurant.place
98. Many-to-Many Example
Part 16: Advanced ORM Concepts
99. Subquery() and OuterRef()
Accessing a reverse one-to-one relationship can raise a related-object exception if the associated object does not exist.
```python
class Publication(models.Model):
title = models.CharField(max_length=100)
class Article(models.Model):
headline = models.CharField(max_length=200)
publications = models.ManyToManyField(Publication)
```
publication = Publication.objects.create(title="Tech Magazine") article =
Article.objects.create(headline="Learning Django") article.publications.add(publication)

article.publications.all() publication.article_set.all() The relevant model instances must be saved before creating a many-to-many relationship.

Use one query inside another query:

```python
from django.db.models import OuterRef, Subquery
latest_article = Article.objects.filter(
reporter=OuterRef("pk")
```
).order_by("-id")
reporters = Reporter.objects.annotate(
latest_headline=Subquery(
latest_article.values("headline")[:1]
)
)

100. Exists()

101. Database Functions

102. select_for_update()

This annotates each Reporter with the headline of their latest Article, based on the example's ordering.

Check whether related records exist:

```python
from django.db.models import Exists, OuterRef
articles = Article.objects.filter(
reporter=OuterRef("pk")
)
reporters = Reporter.objects.annotate(
has_articles=Exists(articles)
)
```

Django database functions perform calculations in the database.

```python
from django.db.models.functions import Lower
students = Student.objects.annotate(
```

lowercase_name=Lower("name")
)
```
Function availability and behavior can vary by database backend. Lock selected rows within a transaction when the database backend supports it:
```python
from django.db import transaction
with transaction.atomic():
account = (
Account.objects
.select_for_update()
.get(pk=1)
)
account.balance -= 100
account.save()
```
103. Signals and save()
save() generally saves one model instance. QuerySet.update() performs a direct database update. bulk_create() and bulk_update() do not follow normal instance-level save behavior.
Part 17: Common Mistakes
104. Forgetting Migrations
105. Confusing get() and filter()
106. Confusing null and blank
107. N+1 Queries
This is useful for concurrency-sensitive operations. Not all database backends, including SQLite, support row-level locking in the same way. Signals can respond to model saves and deletions. However, putting all business logic in signals is often undesirable because it can make the code flow difficult to understand.
Important differences:
Understand which validations, overrides, and signals run for each operation. Problem: A field is added to a model, but the corresponding database column does not exist.
Solution:
```

```python
python manage.py makemigrations
python manage.py migrate
Student.objects.get(pk=1)
```

Expects one object and raises an exception if zero or multiple objects match.

Student.objects.filter(pk=1)

Returns a QuerySet. null relates to database values; blank relates to validation. When accessing related objects inside a loop, check whether select_related() or prefetch_related() is appropriate.

108. Unsafe Raw SQL

109. Unnecessary .all()

110. Incorrect Expectations About Bulk Operations

111. Indexing Every Field

112. External Side Effects Inside Transactions

Part 18: Best Practices

1. Use meaningful model and field names.

2. Enforce data integrity with appropriate UniqueConstraint and CheckConstraint rules.

3. Understand on_delete behavior before selecting a deletion strategy.

4. Keep QuerySet chains readable.

5. Check for N+1 queries when accessing related objects.

6. Consider bulk operations and iterator() when processing large datasets.

7. Measure performance and use explain() before optimizing queries.

8. Use parameterized queries when writing Raw SQL.

9. Plan routers, migrations, and relationships carefully when using multiple databases. Never concatenate untrusted user input directly into SQL strings. Use parameterized queries.

In many cases, you can use a QuerySet directly:

Student.objects.filter(...)

.all() is not inherently wrong, but it can be unnecessary in some expressions. Do not assume update() , bulk_create() , or bulk_update() automatically execute instance-level save() overrides, validation, or signals. Indexes can improve performance, but

unnecessary indexes increase storage and write costs. Use query plans and measurements to guide decisions. Database rollback cannot undo an email or external API call. Consider transaction.on_commit() when the action should happen only after a successful commit.

10. Consider database constraints and locking strategies for concurrency-sensitive operations.

11. Avoid exposing sensitive data in production logs.

12. Never put production credentials in fixtures.

13. Maintain a database backup and recovery strategy.

14. Learn to inspect generated SQL.

15. Consult the official documentation for version-specific and backend-specific behavior.

Part 19: Interview Questions and Quick Revision

Beginner

Intermediate

1. What is Django ORM?

An abstraction that allows developers to work with relational databases using Python objects and QuerySets.

2. What is a Model?

A Python class representing database data and related behavior.

3. What is a Migration?

A versioned mechanism for applying model changes to the database schema.

4. What is the difference between get() and filter() ?

get() expects one object; filter() returns a QuerySet.

5. What is the difference between null=True and blank=True ?

null concerns database values; blank concerns validation.

6. What are select_related() and prefetch_related() ?

Techniques for optimizing queries involving related objects.

7. What is the difference between annotate() and aggregate() ?

annotate() adds calculated values to objects or groups; aggregate() calculates an overall summary.

8. What is an F() expression?

It allows calculations using database field values without first loading those values into Python.

Advanced

9. Why use a Custom Manager?

To create reusable query logic and a model-specific query interface.

10. What is transaction.atomic() ?

It executes database operations within a transaction block.

11. What is the N+1 query problem?

A performance issue in which a main query is followed by additional queries for individual results.

12. What is the difference between QuerySet.update() and save() ?

update() performs a direct SQL update; save() saves a model instance.

13. How do you prevent SQL injection?

Use parameterized queries and avoid concatenating untrusted input into SQL strings.

14. Why use explain() ?

To inspect the database's query execution plan.

15. What is select_for_update() ?

It locks selected rows inside a transaction on supported database backends.

16. What is transaction.on_commit() ?

It runs a callback after a successful database transaction commit.

17. Can foreign keys span multiple databases?

Django does not automatically support ordinary relational foreign keys across separate databases.

18. What are the limitations of bulk_create() ?

It does not follow normal instance-level save() and signal behavior.

19. Why are QuerySets lazy?

They separate QuerySet construction from query execution, allowing Django to avoid executing queries until the results are needed.

20. When should database indexes be used?

When real query patterns and execution plans indicate that an index will be beneficial. Indexes should not be added to every field without considering their costs.

Final Revision Checklist

Models and field types null , blank , default , and choices Meta , constraints, and indexes ForeignKey , OneToOneField , and ManyToManyField related_name and reverse relationships filter() , exclude() , and get() Lookup expressions and Q order_by() , slicing, and values() Create, update, delete, and bulk operations F() , Subquery() , OuterRef() , and Exists() aggregate() and annotate() Search and PostgreSQL full-text search Custom Managers and QuerySets Raw SQL and SQL injection prevention Transactions, savepoints, and on_commit() Multiple databases and database routers Tablespaces QuerySet laziness and caching N+1 problems and query optimization select_related() and prefetch_related() explain() , instrumentation, and logging Fixtures Relationship API Database constraints and concurrency Interview questions Official Django 6.0 Documentation Models: https://docs.djangoproject.com/en/6.0/topics/db/models/ Making queries: https://docs.djangoproject.com/en/6.0/topics/db/queries/ Aggregation: https://docs.djangoproject.com/en/6.0/topics/db/aggregation/ Search: https://docs.djangoproject.com/en/6.0/topics/db/search/ Managers: https://docs.djangoproject.com/en/6.0/topics/db/managers/ Raw SQL: https://docs.djangoproject.com/en/6.0/topics/db/sql/ Transactions: https://docs.djangoproject.com/en/6.0/topics/db/transactions/ Multiple databases: https://docs.djangoproject.com/en/6.0/topics/db/multi-db/ Tablespaces: https://docs.djangoproject.com/en/6.0/topics/db/tablespaces/ Optimization: https://docs.djangoproject.com/en/6.0/topics/db/optimization/
Instrumentation: https://docs.djangoproject.com/en/6.0/topics/db/instrumentation/ Fixtures: https://docs.djangoproject.com/en/6.0/topics/db/fixtures/
Relationship examples: https://docs.djangoproject.com/en/6.0/topics/db/examples/ Recommended study order: Start with Models and Relationships, then learn QuerySets, Aggregation, and Managers. Next, study Transactions, Raw SQL, Multiple Databases, and Optimization. Practicing each topic in your own Django project is one of the most effective ways to learn it.
Django Advanced Notes: Database Transactions,
Custom Managers & ORM Optimization
Table of Contents
1. Database Transactions
2. ACID Properties

3. Transactions in Django

4. Nested Transactions and Savepoints

5. Autocommit and ATOMIC_REQUESTS

6. Custom Model Managers

7. Custom QuerySets and as_manager()

8. Manager vs. QuerySet

9. ORM Optimization

10. The N+1 Query Problem

11. select_related() and prefetch_related()

12. Advanced Prefetch()

13. Aggregation and Annotation

14. Field Selection

15. F() Expressions

16. Subquery() , OuterRef() , and Exists()

17. Bulk Operations

18. Database Indexing

19. Row Locking with select_for_update()

20. QuerySet Lazy Evaluation

21. Pagination and Memory Optimization

22. Query Debugging and Execution Plans

23. Real E-commerce Optimization

24. Common Mistakes

Prepared by: Faruk Ahmed Topics: Django ORM, Database Transactions, Custom Managers, Query Optimization Level: Beginner to Advanced Language: English

25. Interview Questions and Answers

26. Final Revision Checklist

Part 1 — Database Transactions

1. What Is a Database Transaction?

SELECT

INSERT

UPDATE

DELETE

Example: Bank Transfer

1. Deduct 500 from Account A.

2. Add 500 to Account B.

2. SQL Transaction Example

A database transaction is a logical unit of one or more database operations that should succeed together or be rolled back when a failure occurs.

Common database operations include:

A transaction can contain both read and write operations, although read-only queries do not normally modify database data. Suppose we transfer 500 units of currency from Account A to Account B. If both operations succeed, commit the transaction. If an error prevents the transfer from completing, roll back the transaction. Without a transaction, Account A could lose money without Account B receiving it.

```
BEGIN;
UPDATE accounts
SET balance = balance - 500 WHERE id = 1;
UPDATE accounts
SET balance = balance + 500 WHERE id = 2;
COMMIT;
```

COMMIT makes the transaction's changes permanent according to the database's transaction guarantees.

Part 2 — ACID Properties

1. Atomicity

All required operations succeed and the transaction commits. If the transaction is rolled back, its uncommitted changes are undone.

2. Consistency

Account balances must satisfy applicable rules. Foreign keys must reference valid records. Unique fields must not contain duplicate values. Example: Prevent Negative Balances

If the transaction needs to be aborted before commit:

```
ROLLBACK;
```

Important: A transaction does not automatically validate every business rule. Application logic, database constraints, and appropriate error handling are still necessary. ACID describes four important properties of database transactions. Atomicity means that the operations in a transaction are treated as one logical unit.

Example:

Account A = 2000 Account B = 1000 Transfer = 500

After success:

Account A = 1500 Account B = 1500

If the transaction fails and rolls back:

Account A = 2000 Account B = 1000 Consistency means that a successful transaction preserves the database's defined constraints and business rules.

Examples:

```python
from django.db import models
```

3. Isolation

Conditional database updates Row locking Transactions Appropriate transaction isolation levels Database constraints

4. Durability

```python
class Account(models.Model):
balance = models.DecimalField(
max_digits=12,
decimal_places=2,
)
class Meta:
constraints = [
models.CheckConstraint(
condition=models.Q(balance__gte=0),
name="account_balance_nonnegative",
),
]
```

This constraint helps prevent negative balances at the database level on supported backends. Isolation controls how concurrent transactions interact.

Example:

Two customers try to purchase the last available product at the same time. Without appropriate concurrency control, both operations might incorrectly proceed.
Possible techniques include:
The actual behavior depends on the database engine and its configuration. Durability means that successfully committed changes persist according to the database's durability guarantees.
Example:
Payment verified
↓
Transaction committed
ACID Revision
Part 3 — Database Transactions in Django
1. transaction.atomic()
2. Bank Transfer Example
↓
Order marked as paid Once a transaction commits, an ordinary application error does not automatically undo it. A — Atomicity: All or nothing C — Consistency: Preserve defined rules I — Isolation: Control concurrent interactions D — Durability: Committed changes persist Django provides transaction.atomic() to group database operations into a transaction.

```python
from django.db import transaction
with transaction.atomic():
# Perform related database operations.
pass
```

When the block completes successfully, Django commits the outermost transaction, or releases the relevant savepoint if the block is nested. If an exception escapes the block, Django rolls back the relevant transaction or savepoint. Assume an Account model contains a balance field.

```python
from django.db import transaction
from django.db.models import F
```

from .models import Account
@transaction.atomic
def transfer_money(from_id, to_id, amount):
if amount <= 0:
raise ValueError("Amount must be positive.")
```
How It Works
### 1. Validates that the transfer amount is positive.
### 2. Rejects transfers between the same account.
### 3. Deducts money only if the sender has sufficient balance.
### 4. Adds money to the recipient's account.
### 5. Commits both changes if the function completes successfully.
### 6. Rolls back the deduction if an exception escapes the transaction.
if from_id == to_id:
raise ValueError("Accounts must be different.")
updated = Account.objects.filter(
pk=from_id,
balance__gte=amount,
).update(
balance=F("balance") - amount
)
if updated != 1:
raise ValueError(
"Sender account not found or insufficient balance."
)
recipient_updated = Account.objects.filter(
pk=to_id,
).update(
balance=F("balance") + amount
)
if recipient_updated != 1:
raise Account.DoesNotExist(
"Recipient account not found."
```

)

F() expressions perform arithmetic at the database level. The conditional update also avoids relying on a stale balance value read earlier in Python. Production warning: A real financial application needs additional safeguards, including suitable concurrency control, ledger records, idempotency, account-status validation, currency precision, and carefully designed locking.

3. Proper Exception Handling

Part 4 — Nested Transactions and Savepoints

Example A useful pattern is to catch database exceptions outside the relevant atomic() block.

```python
from django.db import DatabaseError, transaction
try:
with transaction.atomic():
perform_database_updates()
except DatabaseError:
# Log the error and handle the failure appropriately.
pass
```

perform_database_updates() is a placeholder for application code. Avoid swallowing errors silently in production. Log relevant information and handle expected exceptions specifically. Important: After a database error breaks a transaction, do not continue issuing queries inside that broken transaction. Exit the relevant atomic() block so that rollback can occur first. Django supports nested atomic() blocks.

```python
from django.db import transaction
with transaction.atomic():
# Outer transaction.
with transaction.atomic():
# Inner transaction / savepoint.
pass
```

A nested atomic() block normally creates a savepoint when savepoints are supported and enabled. A savepoint allows Django to roll back part of a transaction without necessarily rolling back the entire outer transaction. Rolling back to a savepoint is not the same as rolling back the entire transaction. A successful inner block does not independently commit the outer transaction.

Part 5 — Autocommit and ATOMIC_REQUESTS

1. Django Autocommit

```python
from django.db import transaction
with transaction.atomic():
create_order()
try:
with transaction.atomic():
create_optional_coupon()
except ValueError:
print("Coupon operation failed.")
create_order_history()
```

Here, if create_optional_coupon() raises a ValueError , the inner block rolls back and the exception is caught outside that inner block. The outer transaction can continue. If another uncaught exception escapes the outer block, the entire outer transaction is rolled back.

Remember:

Django normally uses autocommit mode.

Account.objects.filter(
pk=1
).update(
balance=1000
)

Outside an explicit transaction, a successful database write is normally committed automatically.

To group related operations:

```python
```

with transaction.atomic():
operation_one()
```
### 2. ATOMIC_REQUESTS
Important Considerations Wrapping every view in a transaction introduces overhead. Middleware processing and template rendering are not necessarily inside the view transaction. Streaming responses can perform work after the view transaction has completed. Transactions should be chosen according to application requirements.
## Part 6 — Custom Model Managers
### 1. What Is a Model Manager?
### 2. What Is a Custom Manager?
operation_two()
operation_three()
In Django settings, ATOMIC_REQUESTS=True for a database can wrap each view's execution in a transaction.
Conceptual configuration:
```python
DATABASES = {
"default": {
# Database configuration...
"ATOMIC_REQUESTS": True,
},
}
```
A Django Model Manager provides the interface used to initiate database queries for a model. The default manager is normally named objects .
```python
Product.objects.all()
Product.objects.filter(
is_active=True
)
Product.objects.get(pk=1)
```
```

Reduces duplicated query code. Makes queries easier to read. Organizes reusable filters. Provides convenient access to common subsets of data.

3. Creating a Custom Manager

A custom manager is a class derived from models.Manager that provides reusable model-querying logic.

Benefits:

```python
from django.db import models
class ActiveProductManager(models.Manager):
def get_queryset(self):
return super().get_queryset().filter(
is_active=True
)
class Product(models.Model):
name = models.CharField(max_length=100)
price = models.DecimalField(
max_digits=10,
decimal_places=2,
)
is_active = models.BooleanField(default=True)
objects = models.Manager()
active = ActiveProductManager()
def __str__(self):
return self.name
```

Usage:

```
all_products = Product.objects.all() active_products = Product.active.all()
```

Suppose the database contains:

4. Custom Manager Methods

ID Name Active 1 Laptop True 2 Mouse False 3 Keyboard True

Then:

Product.objects.all()

Returns all three products.

Product.active.all()
Returns Laptop and Keyboard. Warning: Filtering the default manager can cause records to be unexpectedly excluded
```python
from some operations. Keep the default manager behavior in mind when designing
```
related-object access and framework integrations. You can add methods without overriding get_queryset() .
```python
class ProductManager(models.Manager):
def active_products(self):
return self.get_queryset().filter(
is_active=True
)
def inactive_products(self):
return self.get_queryset().filter(
is_active=False
)
def expensive_products(self, minimum_price=1000):
return self.get_queryset().filter(
price__gt=minimum_price
)
Attach the manager:
```
Part 7 — Custom QuerySets and as_manager()
1. What Is a Custom QuerySet?
```python
class Product(models.Model):
name = models.CharField(max_length=100)
price = models.DecimalField(
max_digits=10,
decimal_places=2,
)
```

is_active = models.BooleanField(default=True)
objects = ProductManager()
Usage:
```
active = Product.objects.active_products() inactive = Product.objects.inactive_products()
expensive = Product.objects.expensive_products(
5000
)
A custom QuerySet lets you create reusable, chainable query methods.
```python
from django.db import models
class ProductQuerySet(models.QuerySet):
def active(self):
return self.filter(is_active=True)
def inactive(self):
return self.filter(is_active=False)
def expensive(self, minimum_price=1000):
return self.filter(
price__gt=minimum_price
)
Attach it to a model:
```
2. Why Use as_manager() ?
3. Manager vs. QuerySet
```python
class Product(models.Model):
name = models.CharField(max_length=100)
price = models.DecimalField(
max_digits=10,
decimal_places=2,
)
is_active = models.BooleanField(default=True)
objects = ProductQuerySet.as_manager()

Usage:

Product.objects.active()

Product.objects.inactive()

Product.objects.active().order_by("name")

Product.objects.active().expensive(5000)

```

The final example returns active products priced above 5000. objects = ProductQuerySet.as_manager() This exposes suitable public QuerySet methods through a Manager, allowing you to write reusable query logic without duplicating every method in a separate manager class. Feature Manager QuerySet Main purpose Start queries Build and filter querysets Typical access Product .object s Product .objects .all() Golden Rule Use a Manager to provide a convenient query entry point. Use a QuerySet for reusable, chainable filtering logic. Use a service layer for complex multi-step business workflows when appropriate.

Part 8 — The N+1 Query Problem

1. What Is ORM Optimization?

Query count Query execution time Data transferred from the database Memory consumption Database workload Response latency Correctness under concurrency

2. Example Models

Custom methods Supported Supported Chaining Depends on the method Naturally supports query chaining Created by models. Manager models. QuerySe t Django ORM optimization improves database performance by reducing unnecessary queries, excessive data retrieval, expensive database work, and unnecessary Python processing. Optimization should consider:

```python
from django.db import models
```

3. Inefficient Query Pattern

Why Is It a Problem? More database round trips Slower pages and APIs Increased database workload Greater use of database connections and server resources

Part 9 — select_related()

```python
class Author(models.Model):
```
```

name = models.CharField(max_length=100)
class Book(models.Model):
title = models.CharField(max_length=200)
author = models.ForeignKey(
Author,
on_delete=models.CASCADE,
related_name="books",
)
```
books = Book.objects.all()
for book in books:
print(book.title, book.author.name)
If author relationships are not already loaded, a typical query pattern is:
1 query → Books N queries → Authors Potential total: N + 1 queries If 100 books are retrieved, the code may generate 101 queries. If several books share an author, relationship caching may reduce the actual number of queries, but the N+1 problem can still occur. The primary solutions are select_related() and prefetch_related() .
select_related() retrieves related objects using SQL joins. It is primarily designed for single-valued relationships.
ForeignKey OneToOneField Example Multiple Relationships When Should You Use It? A book's author A product's category An order's customer A payment's order and user Another suitable single-valued relationship
Typical relationships:
books = Book.objects.select_related("author")
for book in books:
print(book.title, book.author.name)
Django can retrieve books and their authors in one SQL query.
Conceptual SQL:
SELECT
book.id,
book.title,
book.author_id,
author.id,
```

author.name
FROM book INNER JOIN author
ON book.author_id = author.id;
The actual SQL depends on the model configuration and database backend. payments = Payment.objects.select_related(
"order__user"
)
This can load the payment, its order, and the order's user through joins, assuming those relationship fields exist.
Use select_related() when you need:
Part 10 — prefetch_related()
ManyToManyField Reverse ForeignKey Other suitable multi-valued relationships
Example: Product Images Avoid adding unnecessary joins to long relationship chains. Extra joins can increase query complexity and the amount of data transferred.
prefetch_related() is primarily used for relationships that can contain multiple related objects.
Common use cases:
Django generally executes separate queries and associates the results in Python.
```python
class Product(models.Model):
name = models.CharField(max_length=100)
class ProductImage(models.Model):
product = models.ForeignKey(
Product,
on_delete=models.CASCADE,
related_name="images",
)
image = models.ImageField(
upload_to="products/"
)
Without optimization:
```
products = Product.objects.all()

for product in products:
for image in product.images.all():
print(image.image.url)
A typical query pattern for 100 products is:
1 query → Products 100 queries → Images Many-to-Many Example Multiple Prefetch Operations
Part 11 — Advanced Prefetch()
Potential total: 101 queries
With optimization:
products = Product.objects.prefetch_related(
"images"
)
for product in products:
for image in product.images.all():
print(image.image.url)
A typical query pattern becomes:
1 query → Products 1 query → Images Potential total: 2 queries Actual query counts vary with batching, relationships, and backend behavior. books = Book.objects.prefetch_related("authors")
for book in books:
print(book.title)
for author in book.authors.all():
print(author.name)
Django generally retrieves the books and authors separately, then associates them in Python. products = Product.objects.prefetch_related(
"images",
"reviews",
)
This assumes both images and reviews are valid related names. Filter related objects Order related objects Store prefetched results in a separate attribute Optimize nested related queries
1. Filtered Prefetch
2. Using to_attr

Prefetch() provides greater control over related querysets.
It is useful when you need to:
Suppose a product has many reviews, but only reviews rated 4 or higher are needed.
```python
from django.db.models import Prefetch
products = Product.objects.prefetch_related(
Prefetch(
"reviews",
queryset=Review.objects.filter(
rating__gte=4
),
)
)
```
This prefetches only reviews rated 4 or higher into the related-object cache.
However, a later query such as:
product.reviews.filter(rating=5) may execute another query because it uses a different filter. When filtered data should be clearly separated from the normal related manager, use to_attr . products = Product.objects.prefetch_related(
Prefetch(
"reviews",
queryset=Review.objects.filter(
rating__gte=4
),
to_attr="good_reviews",
)
)
Clearly separates filtered results. Avoids confusion with the normal related manager.
Makes iteration straightforward.
3. Nested Prefetch
for product in products:
for review in product.good_reviews:
print(review.rating)

Here, good_reviews is a Python list.

Benefits:

Suppose each order has order items and each item references a product.

```python
class OrderItem(models.Model):
order = models.ForeignKey(
"Order",
on_delete=models.CASCADE,
related_name="items",
)
product = models.ForeignKey(
Product,
on_delete=models.CASCADE,
)
A basic query:
orders = Order.objects.prefetch_related(
"items__product"
)
An alternative combines prefetching with select_related() :
orders = Order.objects.prefetch_related(
Prefetch(
"items",
queryset=OrderItem.objects.select_related(
"product"
),
)
)
```

4. select_related() vs. prefetch_related()

Simple Rule This generally retrieves orders separately from order items joined with their products. The most efficient option depends on the query pattern and data volume.

Feature select_ related(

)

prefetc h_relate d() Main use ForeignKe y and OneToOn e ManyToM any and reverse ForeignKe y Technique SQL joins Separate queries and Python associatio n Query count Often one main query Usually multiple queries Best for Single related objects Related collections Custom filtered relations Not its main purpose Supported through Prefetc h()

Part 12 — Aggregation and Annotation

1. annotate()

Example: Count Reviews

2. aggregate()

ForeignKey / OneToOne

↓

select_related() ManyToMany / Reverse ForeignKey

↓

prefetch_related() Filtered related objects

↓

Prefetch() annotate() adds calculated values to each object in a QuerySet.

```python
from django.db.models import Count
products = Product.objects.annotate(
review_count=Count("reviews")
)
for product in products:
print(product.name, product.review_count)
```

Here, reviews must be a valid related name. Django can perform the count at the database level, avoiding a separate count query for each product. aggregate() calculates summary values for the entire QuerySet.

```python
from django.db.models import Avg, Count
summary = Product.objects.aggregate(
average_price=Avg("price"),
total_products=Count("id"),
)
```

3. annotate() vs. aggregate()

4. Aggregation Warning

Part 13 — Field Selection

1. only()

print(summary)

Conceptual result:

{

"average_price": 2500,

"total_products": 20,

}

Actual results depend on the data and field types. annotate() → Calculated value for each object. aggregate() → Summary values for the whole QuerySet. Joining multiple one-to-many relationships can multiply result rows and produce incorrect counts or sums.

For example:

```python
Product.objects.annotate(
review_count=Count(
"reviews",
distinct=True,
)
)
```

distinct=True may prevent duplicate review counts caused by joins, but it is not a universal fix for every aggregation problem. Inspect the generated SQL and verify the results. Loads specified fields initially and defers other fields until they are accessed.

products = Product.objects.only(

"id",

"name",

)

2. defer()

3. values()

Accessing a deferred field later may execute another query. product =
Product.objects.only(
"id",
"name",
).get(pk=1) print(product.name) The name field is already loaded.
If description was deferred:
print(product.description) Django may issue another query to retrieve it. Excludes specified fields from the initial query. products = Product.objects.defer(
"description"
)
This can help when large text fields are not needed. Returns dictionaries containing selected field values. products = Product.objects.values(
"id",
"name",
"price",
)
Conceptual result:
[
{
"id": 1,
"name": "Laptop",
"price": 50000,
},
{
"id": 2,
"name": "Phone",
4. values_list()
5. values_list(flat=True)
Comparison
"price": 20000,
},
]
Returns tuples by default. products = Product.objects.values_list(

"name",
"price",
)
Use flat=True when selecting exactly one field. names = Product.objects.values_list(
"name",
flat=True,
)
Conceptual result:
["Laptop", "Phone", "Keyboard"] Method Output values(
)
Dictionari es values_ list() Tuples values_ list(fla t=True) A sequence of singlefield values only() Model instances
Part 14 — F() Expressions
1. Decrease Stock
2. Conditional Stock Update
with deferred fields defer() Model instances with specified fields deferred
If only identifiers are needed:
product_ids = Product.objects.values_list(
"id",
flat=True,
)
This can avoid constructing full model instances. An F() expression references a database field in a query, allowing calculations without first loading the current field value into Python.
```python
from django.db.models import F
Product.objects.filter(
pk=1
).update(
stock=F("stock") - 1
)
updated = Product.objects.filter(
pk=1,
```

stock__gt=0,
```
Benefits Avoids loading the old field value into Python. Performs arithmetic at the database level. Reduces common read-modify-write race conditions. Works well with bulk updates.
## Part 15 — Subquery() and OuterRef()
Example: Latest Review Rating ).update(
stock=F("stock") - 1
)
if updated == 0:
print("Product not found or stock unavailable.")
The condition and stock update occur as part of the same database update operation. Important: F() expressions do not solve every concurrency problem. More complex workflows may require transactions, locking, and database constraints. A Subquery() embeds another query inside a larger query. OuterRef() references a field from the outer query. Assume each review has a product , rating , and created_at field.
```python
from django.db.models import OuterRef, Subquery
latest_review = (
Review.objects
.filter(product=OuterRef("pk"))
.order_by("-created_at", "-pk")
)
products = Product.objects.annotate(
latest_rating=Subquery(
latest_review.values("rating")[:1]
)
)
Usage:
```
Part 16 — Exists()
Example: Check Whether a Product Has Reviews Retrieve Only Products with Reviews for product in products:
```

print(
product.name,
product.latest_rating,
)
Each product receives the rating of its latest review. If a product has no reviews, latest_rating is normally None . Ordering by -created_at and then -pk provides a deterministic tiebreaker when timestamps are equal. Exists() checks whether a subquery contains at least one matching record.
```python
from django.db.models import Exists, OuterRef
reviews = Review.objects.filter(
product=OuterRef("pk")
)
products = Product.objects.annotate(
has_review=Exists(reviews)
)
for product in products:
print(
product.name,
product.has_review,
)
Conceptual output:
```
Laptop True Phone True Keyboard False The actual output depends on the database contents.
Part 17 — Bulk Operations
1. bulk_create()
Inefficient Pattern Optimized Pattern Important Limitations Does not call each instance's save() method. products_with_reviews = Product.objects.filter(
Exists(reviews)
)
Use an annotation when you need a Boolean value for every product. Use filtering when you only need products that have reviews. bulk_create() inserts multiple objects

efficiently.
for i in range(1000):
Product.objects.create(
name=f"Product {i}",
price=100,
)
This generally executes many separate insert operations. products = [
Product(
name=f"Product {i}",
price=100,
)
for i in range(1000)
]
Product.objects.bulk_create(
products,
batch_size=500,
)
With batch_size=500 , Django divides the inserts into batches as necessary. It is not guaranteed to execute everything in a single SQL query.
Bulk creation generally:
Does not trigger normal per-instance save signals. Has backend-specific behavior regarding returned primary keys and conflict handling. Requires consideration of memory usage for very large lists.
2. bulk_update()
Limitations Loading objects may require an initial query. The update may require multiple SQL batches. Each object's save() method is not called. Normal per-instance save signals are not triggered. Memory use can be significant for large collections.
3. Prefer update() for Simple Database-Side Changes
bulk_update() updates selected fields on multiple existing objects. products = list(
```python
Product.objects.filter(
is_active=True
)
```

)
for product in products:
product.price += 10
Product.objects.bulk_update(
products,
["price"],
batch_size=500,
)
```
This example modifies active products in Python, then updates their prices in bulk.
If the goal is simply to increase every active product's price:
```python
Product.objects.filter(
is_active=True
).update(
price=F("price") + 10
)
```
Bulk Operation Summary
Part 18 — Database Indexing
1. What Is a Database Index?
WHERE JOIN ORDER BY This performs the arithmetic at the database level and avoids loading every object into Python. Requirem ent Recomm ended API Insert many objects bulk_cr eate() Update selected fields on existing objects bulk_up date() Update matching rows directly QuerySe t.update () Delete matching rows QuerySe t.delete () A database index helps the database locate matching records efficiently.
Indexes can improve suitable queries involving:
Unique lookups
2. Single-Field Index
3. Composite Index
However, indexes consume storage and require maintenance during inserts, updates, and deletes.
```python

class Product(models.Model):
name = models.CharField(
max_length=100,
db_index=True,
)
```
Use db_index=True when the field's query patterns justify an index.
If email addresses must be unique:
email = models.EmailField(
unique=True
)
A uniqueness constraint normally creates an appropriate database index, depending on the backend.
```python
class Product(models.Model):
name = models.CharField(max_length=100)
price = models.DecimalField(
max_digits=10,
decimal_places=2,
)
class Meta:
indexes = [
models.Index(
fields=["name", "price"],
name="product_name_price_idx",
),
]
```
This creates a composite index over name and price .
It may help queries such as:
4. Indexing Best Practices
Frequent filtering Common joins Important ordering patterns Uniqueness requirements Actual query plans
```

Part 19 — Row Locking with select_for_update()

Example: Reserve Product Stock

```python
Product.objects.filter(
name="Laptop",
price__gt=1000,
)
```

The order of fields matters. An index starting with name may also help queries filtering only on name , but it may not be suitable for queries filtering only on price .

Create indexes based on:

Avoid creating an index on every field. Golden rule: Measure query performance and inspect the execution plan before adding indexes. On supported databases, select_for_update() requests locks on selected rows until the relevant transaction ends. It is useful when concurrent operations need to coordinate changes to the same records.

```python
from django.db import transaction
def reserve_stock(product_id, quantity):
if quantity <= 0:
raise ValueError(
"Quantity must be positive."
)
with transaction.atomic():
product = (
Product.objects
.select_for_update()
```

Important Considerations Use row locking inside a transaction. Database backend support varies. SQLite does not provide the same row-level locking behavior as PostgreSQL. Avoid holding locks while performing slow external operations. Consider deadlocks and lock contention.

Part 20 — QuerySet Lazy Evaluation

.get(pk=product_id)

)
if product.stock < quantity:
raise ValueError(
"Insufficient stock."
)
product.stock -= quantity
product.save(
update_fields=["stock"]
)
For simple stock deductions, a conditional update can be a useful alternative:
updated = Product.objects.filter(
pk=product_id,
stock__gte=quantity,
).update(
stock=F("stock") - quantity
)
if updated == 0:
raise ValueError(
"Product not found or insufficient stock."
)
This approach atomically checks and decreases stock in one update statement, but more complex business workflows may still require a transaction. Django QuerySets are generally lazy. Creating a QuerySet normally does not immediately execute the database query. Common Evaluation Triggers exists() vs. count()
Part 21 — Pagination and Memory Optimization
1. Pagination
products = Product.objects.filter(
is_active=True
)
The query generally executes when the QuerySet is evaluated.
for product in products:
print(product.name)

list(products) len(products) bool(products) Other triggers include accessing individual results and converting the QuerySet to a list.

To check whether at least one record exists:

if Product.objects.filter(

is_active=True

).exists():

print("Active products found.")

To count records:

count = Product.objects.filter(

is_active=True

).count() When you only need to check existence, exists() is usually more appropriate than count() > 0 . A QuerySet that has already been fully evaluated and cached may affect whether another database query is needed, so behavior depends on the QuerySet's state. Pagination retrieves data in smaller portions instead of displaying an entire dataset at once. Important Considerations Use stable ordering when paginating. Offset-based pagination can become inefficient for very large page numbers. Cursor-based pagination may be appropriate for large datasets or frequently changing data.

2. iterator()

Part 22 — Query Debugging and Execution Plans

1. connection.queries

```python
from django.core.paginator import Paginator
```

products = Product.objects.order_by("id")

paginator = Paginator(products, 20)

page = paginator.get_page(1)

for product in page:

print(product.name)

This displays up to 20 products on the selected page. For large QuerySets, iterator() can reduce memory use by bypassing the normal QuerySet result cache.

for product in Product.objects.iterator(

chunk_size=1000

):

process_product(product)

process_product() is a placeholder. Actual memory and query behavior depends on the database driver, backend, and related-object access patterns. If you need related objects, consider whether prefetching or another batching strategy is appropriate. During development, query logging can help inspect executed queries.

```python
from django.db import connection
```

Limitations It is not a substitute for production query monitoring. Logs may contain sensitive query information. Behavior depends on the database connection and configuration. Large query logs can become difficult to analyze.

2. Django Debug Toolbar

SQL queries Duplicate queries Query timings N+1 query problems Slow view behavior

3. QuerySet.explain()

Table scans Index usage Join strategies Estimated row counts Other execution details

Part 23 — Real E-commerce Optimization Example

print(connection.queries) This is useful only when the relevant query logging is enabled, commonly through DEBUG=True .

Django Debug Toolbar can help identify:

Use it in development and configure access appropriately. explain() displays the database execution plan for a QuerySet. queryset = Product.objects.filter(

is_active=True

)

print(queryset.explain())

Depending on the database backend, the plan may reveal:

Compare query plans before and after optimization. The output format and available options depend on the backend. Product Category ProductImage Review Product has a ForeignKey to Category. Product images use the reverse name images . Product reviews use the reverse name reviews . Optimized Query What Has Been Optimized?

1. select_related("category") loads category data through a SQL join.

2. prefetch_related("images") retrieves product images efficiently.

3. Prefetch() retrieves only reviews rated 4 or higher into good_reviews .

Suppose an e-commerce application has:

Assumptions:

```python
from django.db.models import Count, Prefetch
products = (
Product.objects
.select_related("category")
.prefetch_related(
"images",
Prefetch(
"reviews",
queryset=Review.objects.filter(
rating__gte=4
),
to_attr="good_reviews",
),
)
.annotate(
review_count=Count(
"reviews",
distinct=True,
)
)
)
```

4. annotate() calculates the review count in the database.
5. distinct=True helps avoid duplicate review counts caused by certain joins.
Template Example
Part 24 — Common ORM Mistakes
Mistake 1: Using select_related() for Every Relationship Important: Complex aggregations should still be verified. Inspect the SQL and check the results, especially when multiple one-to-many relationships are involved. {% for product in products %}

{{ product.name }}

Category: {{ product.category.name }}

Review Count: {{ product.review_count }}

{% for image in product.images.all %}

src="{{ image.image.url }}"
alt="{{ product.name }}"
>
{% endfor %}

Good Reviews

{% for review in product.good_reviews %}

Rating: {{ review.rating }}

{% endfor %}
{% endfor %} This template assumes the model relationships and fields match the example. Remember that prefetching every related object can increase memory use. Fetch only what the page actually needs. select_related() is primarily for single-valued relationships. Mistake 2: Filtering Again After Prefetching Mistake 3: Assuming Bulk Operations Call save() Mistake 4: Indexing Every Field Mistake 5: Performing Critical Updates Without Transactions Mistake 6: Catching Every Exception Use prefetch_related() for many-to-many relationships and reverse foreign-key collections. products = Product.objects.prefetch_related(

"reviews"
)
Then:
product.reviews.filter(
rating__gte=4
)
The additional filter generally executes another query because it differs from the prefetched .all() result. Use Prefetch() with a filtered queryset if you only need certain related objects. bulk_create() and bulk_update() do not follow the normal per-instance save() lifecycle. Verify your application logic if it depends on model methods or signals. Indexes increase storage and write overhead. Choose indexes according to real query patterns and execution plans. Use transaction.atomic() when dependent database operations must succeed or fail together.
Avoid:
try:
perform_operation()
except:
pass
This hides errors and can create data-integrity problems. Catch expected exceptions and handle failures deliberately. Mistake 7: Optimizing Only Query Count Query execution time Execution plans Rows scanned Data transferred Memory consumption Lock contention Correctness
Part 25 — Interview Questions and Answers
Q1. What is a database transaction? Q2. What are ACID properties? Atomicity: All or nothing. Consistency: Preserve database rules. Isolation: Control concurrent transaction interactions. Durability: Committed changes persist. Q3. What is transaction.atomic() ? Q4. What is a Django Model Manager? Q5. What is the difference between Manager and QuerySet? Q6. What is the N+1 query problem? A low query count does not guarantee a fast query.
Consider:
A transaction is a logical unit of database operations that are committed together or rolled back when necessary. It is a Django context manager and decorator for grouping database operations into a transaction. A Manager provides the interface for initiating

and organizing model queries. A Manager starts queries, while a QuerySet represents a collection of query results and provides filtering and chaining operations. It occurs when one query retrieves a collection of objects and additional queries are executed for related data for each object. Q7. What is the difference between select_related() and prefetch_related() ? Q8. What is Prefetch() ? Q9. What is the difference between annotate() and aggregate() ? Q10. What is an F() expression? Q11. What is Exists() ? Q12. What is bulk_create() ? Q13. What is bulk_update() ? Q14. What is a database index? Q15. How do you debug slow Django queries? Django Debug Toolbar Query logging in development QuerySet.explain() Database monitoring tools select_related() uses SQL joins for single-valued relationships. prefetch_related() generally uses separate queries and associates related collections in Python. Prefetch() customizes prefetching, including filtering related objects and storing results in a separate attribute using to_attr . annotate() adds calculated values to individual objects. aggregate() returns summary values for the entire QuerySet. An F() expression references a database field so calculations can occur at the database level without first reading the old value into Python. Exists() is an ORM expression that checks whether a subquery contains at least one matching row. It is a Django API for inserting multiple model objects in bulk. It is a Django API for updating selected fields on multiple existing objects in bulk. An index is a database structure that helps locate records efficiently, at the cost of extra storage and write overhead.

Use:

Performance measurements before and after optimization Q16. What is the difference between only() and defer() ? Q17. What is the difference between values() and values_list() ? Q18. What is select_for_update() ? Q19. What is QuerySet lazy evaluation? Q20. Why use a custom QuerySet?

Part 26 — Final Revision Checklist

Database Transactions Understand transactions and ACID. Use transaction.atomic() correctly. Understand rollback and nested savepoints. Understand autocommit and ATOMIC_REQUESTS . Handle database errors appropriately. Protect concurrent updates using suitable techniques. Custom Managers and QuerySets Understand Django Model Managers. Create custom manager methods. Create custom QuerySet methods. Understand as_manager() . only() specifies the fields to load initially. defer() specifies the fields to postpone loading. values() returns dictionary-like rows. values_list()

returns tuples, or a flat sequence when flat=True is used with one field. It requests row locks for selected records on supported database backends, helping coordinate concurrent operations inside a transaction. A QuerySet generally delays executing its database query until the result is evaluated, such as through iteration or conversion to a list. A custom QuerySet provides reusable, chainable query methods that keep filtering logic organized and reduce duplication. Choose the default manager carefully. Keep complex business workflows organized. ORM Optimization Identify N+1 query problems. Choose between select_related() and prefetch_related() . Use Prefetch() for filtered related data. Understand annotate() and aggregate() . Use F() , Subquery() , OuterRef() , and Exists() . Understand the limitations of bulk operations. Use only() , defer() , values() , and values_list() appropriately. Use pagination and batching for large datasets. Use iterator() when it suits the workload. Database Performance Create indexes based on real workloads. Inspect execution plans with explain() . Use Django Debug Toolbar in development. Measure performance before and after changes. Consider query count, execution time, memory, and data correctness together.

Final Advice

The most important habit in Django ORM optimization is to avoid optimizing based on assumptions.

Follow this process:

1. Identify the performance problem.

2. Inspect the SQL queries.

3. Measure query count and execution time.

4. Examine the execution plan.

5. Apply the appropriate optimization.

6. Measure performance again.

7. Verify correctness and concurrency behavior.

A well-optimized Django application balances database performance, memory usage, maintainability, data integrity, and correctness under concurrent requests.

Django ORM __ Lookups Cheat Sheet with

Example Models Example Models

```python
from django.db import models
class Author(models.Model):
```

```
name = models.CharField(max_length=100) age = models.IntegerField() is_active =
models.BooleanField(default=True)
```python
class Tag(models.Model):
```
name = models.CharField(max_length=50)
```python
class Entry(models.Model):
```
title = models.CharField(max_length=200) pub_date = models.DateField() published =
models.BooleanField(default=False) author = models.ForeignKey(Author,
on_delete=models.CASCADE) tags = models.ManyToManyField(Tag) price =
models.DecimalField(max_digits=6, decimal_places=2)
Date / Time lookups
Entry.objects.filter(pub_date__year=2026)
Entry.objects.filter(pub_date__month=1)
Entry.objects.filter(pub_date__day=29)
Entry.objects.filter(pub_date__week=5)
Entry.objects.filter(pub_date__week_day=3) # Sunday=1, Monday=2,...
Entry.objects.filter(pub_date__quarter=1)
Entry.objects.filter(pub_date__gte='2026-01-01')
Entry.objects.filter(pub_date__lte='2026-12-31')
Entry.objects.filter(pub_date__range=['2026-01-01','2026-12-31'])
String / Text lookups
Entry.objects.filter(title__exact='Python')
Entry.objects.filter(title__iexact='python')
Entry.objects.filter(title__contains='Django')
Entry.objects.filter(title__icontains='django')
Entry.objects.filter(title__startswith='The')
Entry.objects.filter(title__istartswith='the')
Entry.objects.filter(title__endswith='Entry')
Entry.objects.filter(title__iendswith='entry')
```

Entry.objects.filter(title__regex=r'^Py.*$')

Entry.objects.filter(title__iregex=r'^py.*$')

Numeric lookups

Entry.objects.filter(id__gt=5)

Entry.objects.filter(id__gte=5)

Entry.objects.filter(id__lt=10)

Entry.objects.filter(id__lte=10)

Entry.objects.filter(id__exact=7)

Entry.objects.filter(id__in=[1,3,5])

Entry.objects.filter(price__range=[100,500])

Boolean / Null lookups

Entry.objects.filter(published__exact=True)

Entry.objects.filter(published__exact=False)

Entry.objects.filter(pub_date__isnull=True)

Entry.objects.filter(pub_date__isnull=False)

Relational lookups (ForeignKey / OneToOne / ManyToMany)

Entry.objects.filter(author__name='Faruk')

Entry.objects.filter(author__age__gte=18)

Entry.objects.filter(author__is_active=True)

Entry.objects.filter(tags__name__icontains='python')

Miscellaneous / PostgreSQL specific

Entry.objects.filter(title__length=10)

Entry.objects.filter(title__startswith='A')

Entry.objects.filter(title__contains='abc')

Combining lookups

Multiple conditions

Entry.objects.filter(pub_date__year=2026, title__icontains='Python')

Exclude

Entry.objects.exclude(pub_date__year=2026)

Chaining

Entry.objects.filter(pub_date__year=2026).filter(title__icontains='Python')

Tip: All lookups are used in the format field__lookup=value . You can chain multiple filters and use .exclude() for negation.